Getting Started Guide

Install, configure and verify your Blackie Networks WireGuard server

Blackie Networks

WireGuard VPN Management Platform · docs.blackie-networks.com
Generated 2026-08-18

What is Blackie Networks?

Blackie Networks is a self-hosted WireGuard VPN management platform. It runs a WireGuard server alongside a REST API and MongoDB, and adds account management, billing, and MikroTik RouterOS auto-provisioning on top so that end users can rent a managed VPN-connected router without touching the command line.

System architecture

The backend runs as three cooperating services:

- wireguard — a privileged container that runs only the WireGuard kernel interface (wg0). No HTTP traffic touches it.
- wireguard-api — the Node/Express REST API. It shares the wireguard container's network namespace (network_mode: "service:wireguard"), so it can manage peers with the wg / wg-quick tools without needing its own privileged access.
- mongo — MongoDB, used to persist clients, users, routers, subscriptions and billing records.

> **NOTE**
> Splitting the VPN interface from the API process means an API restart or crash never drops active VPN tunnels — the wireguard container keeps routing traffic independently.

Prerequisites

- Docker and Docker Compose installed on the host.
- A public IP or DNS endpoint the WireGuard interface will bind to.
- A WireGuard private key (generate one with wg genkey if you don't already have one).

Step 1 — Configure environment variables

Create a .env file in wireguard-server/ with at least the following keys:

```
WIREGUARD_PRIVATE_KEY=<output of "wg genkey">
MONGO_URI=mongodb://mongo:27017/wireguard
SERVER_ENDPOINT=your-server-ip-or-domain:51820
JWT_SECRET=<a long random string>
FRONTEND_URL=https://app.yourdomain.com
MIKROTIK_SYSTEM_USERNAME=wgmonitor
MIKROTIK_SYSTEM_PASSWORD=<a strong password>
ROUTER_MONTHLY_PRICE=<price in your billing currency>
WG_ENABLED=true
```

> **NOTE**
> .env holds live secrets. Keep it out of version control and never commit it.

Step 2 — Start the stack

The easiest path is the helper script, which builds the image, stops any existing container and starts everything with the correct privileged flags:

```
# Linux / macOS
chmod +x docker-run.sh
./docker-run.sh

# Windows
docker-run.bat
```

Or start the full multi-container stack directly with Compose:

```
docker-compose up -d

# Tail logs for each service
docker-compose logs -f wireguard       # VPN interface
docker-compose logs -f wireguard-api   # REST API
docker-compose logs -f mongo           # database
```

Step 3 — Create an admin account

Bootstrap the first administrator directly against the running API container:

```
node scripts/create-admin.js admin@yourdomain.com "Admin Name" a-strong-password
```

Step 4 — Run the frontend dashboard

The React/Vite admin dashboard lives in frontend/. Point it at your API with VITE_API_URL, then install and run it:

```
cd frontend
echo "VITE_API_URL=http://your-server:5000" > .env
npm install
npm run dev   # serves on http://localhost:5173
```

Step 5 — Verify everything is working

- GET /api/health should return a healthy status from the API.
- docker exec -it wireguard wg show should list the wg0 interface with zero or more peers.
- Log into the dashboard at http://localhost:5173 with the admin account created in Step 3.

Default network configuration

Setting	Value
WireGuard port	UDP 51820
API port	TCP 5000
VPN subnet	10.0.0.0/24
Server VPN IP	10.0.0.1

Where to go next

- To connect a MikroTik router as a managed peer, see the MikroTik Configuration Guide.
- To integrate against the REST API directly (clients, routers, billing, admin), see the API Documentation.